

Ensemble methods

Introduction

- In this lecture we will cover two ensemble strategies: **parallel** and **sequential**.
- We will cover:
 - Recap: decision trees and bias-variance trade-off
 - Parallel ensemble: Bagging and random forests
 - Sequential: AdaBoost, overlook of other boosting method





Recap: decision trees

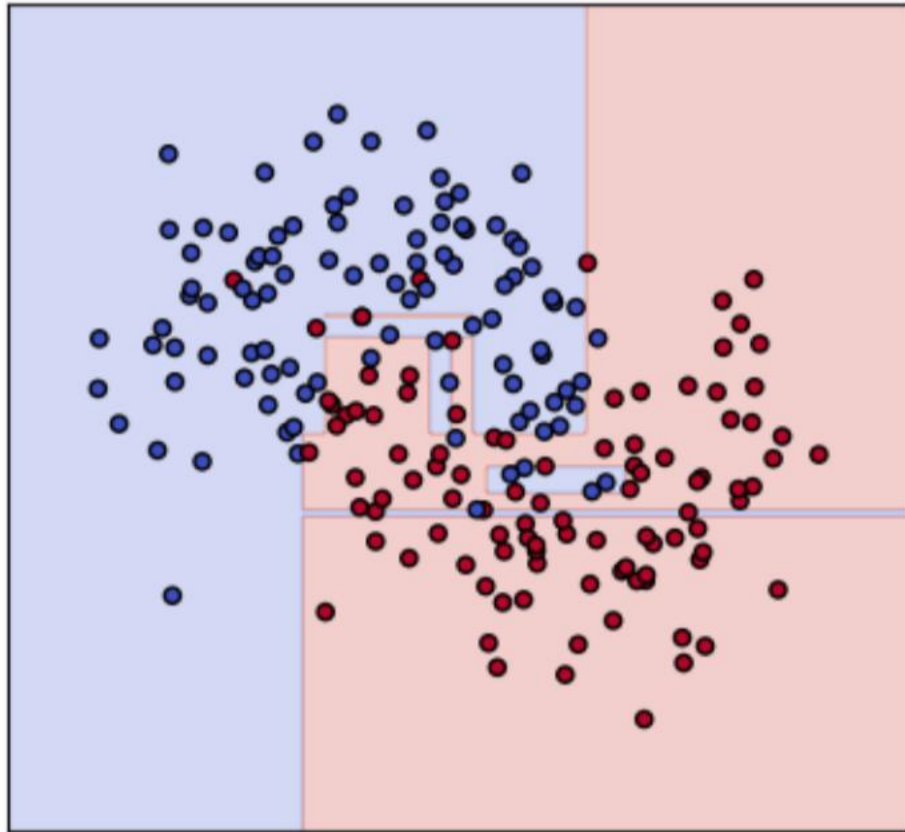
- Greedy, recursive partitioning of the feature space
- Deep tree: low bias, high variance
- Shallow tree: high bias, low variance

Ensambles aim at reducing the bias-variance trade-off 🏆

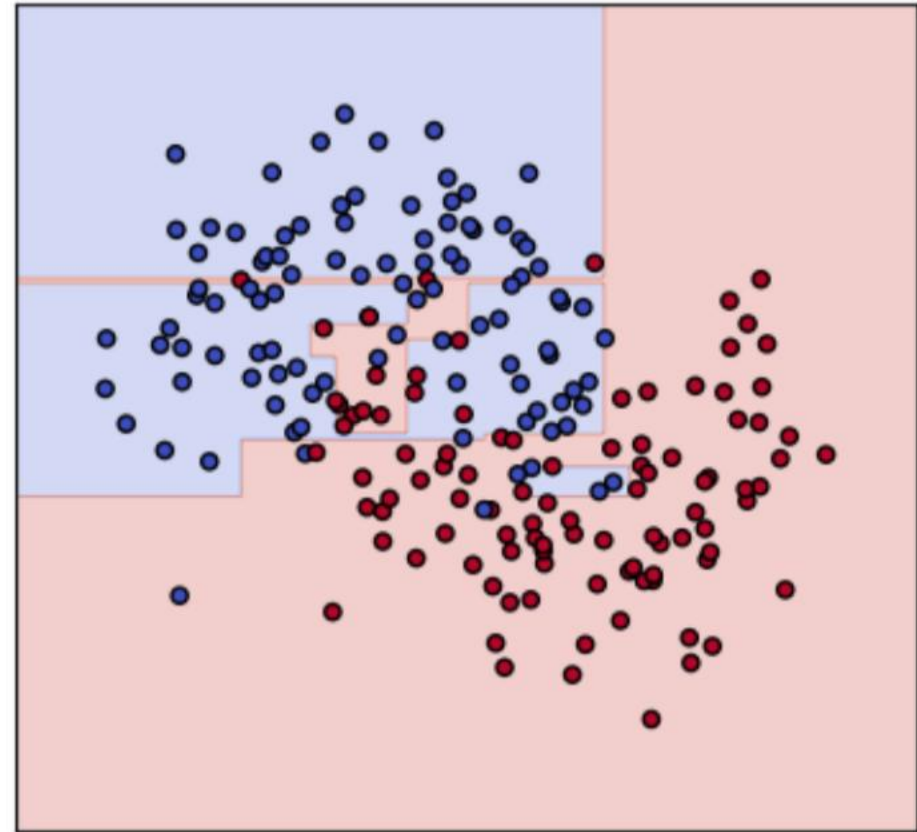
- Bagging → reduces the variance: perfect for unstable good learners
- Boosting → reduces the bias: perfect in case of weak learners



A single deep tree: small data changes, very different boundary
bootstrap resample, seed=1



bootstrap resample, seed=2





How can we reduce variance? → **AVERAGING**

- Consider B predictors z_B , each with variance σ^2 , pairwise correlation ρ :

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B z_b\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

- The $\frac{1}{B}$ term vanishes as $B \rightarrow \infty$
- The $\rho\sigma^2$ term does not: correlated predictors put a floor on how much averaging helps
- Two ways to lower the variance: increase B (bagging), or/and decrease ρ (random forest)



Bagging (Breiman, 1996)

AIM: reducing the variance of unstable learners by averaging

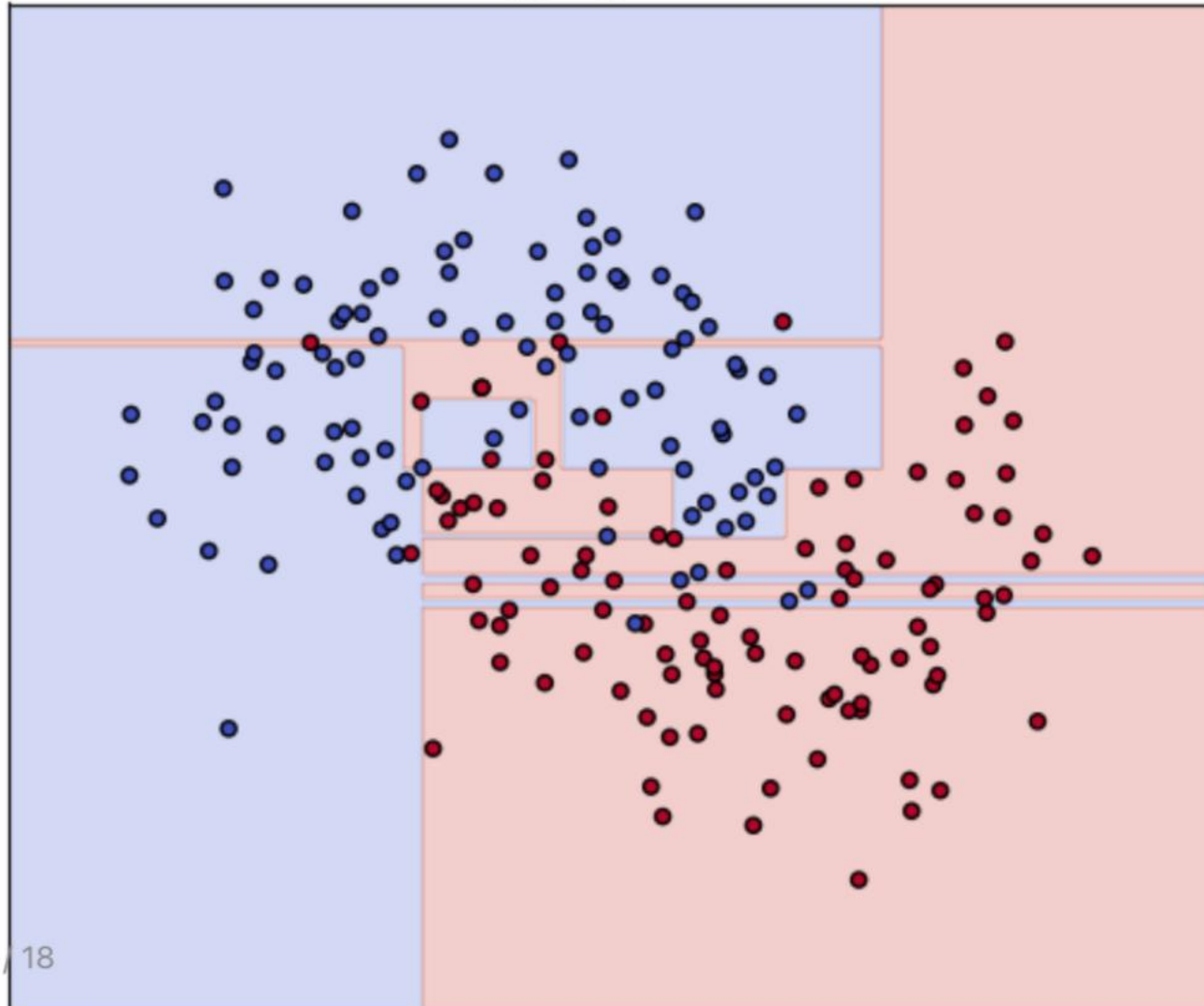
how $\rightarrow$ averaging predictors!

- Bootstrap: draw N samples **with replacement** from the N -sample training set
- Fit one predictor per bootstrap sample, B total
- Predict: mean (regression) or majority vote (classification)
- Each bootstrap leaves out $\approx 1/e \approx 36.8\%$ of points
 - Free validation set: **out-of-bag (OOB) error**

number of trees



Bagging, 1 tree



6 / 18





Bagging beyond trees: KDE Naive Bayes

Bagging is not restricted to decision trees. We can also create an ensemble of **KDE-based Naive Bayes classifiers**.

For each ensemble member $b = 1, \dots, B$:

1. Draw a **bootstrap sample** $D^{*(b)}$ from the training set.
2. Sample a kernel width h_b around a reasonable reference value.
3. Fit a KDE Naive Bayes classifier using $D^{*(b)}$ and h_b .

$$\hat{p}_b(x_j \mid y = c) = \frac{1}{n_c h_b} \sum_{i: y_i = c} K\left(\frac{x_j - x_{ij}}{h_b}\right).$$



- Helps most for **unstable** learners (deep trees);
- Trees only depend on their own bootstrap sample: fits in parallel;
- Does not reduce bias - still limited by how well one tree can fit;
- if the predictors are strongly correlated (many trees can choose the same predictor when there is a strong one) $\rightarrow \rho$ is dominating and averaging does not reduce the variance.

Random forests (Breiman, 2001)

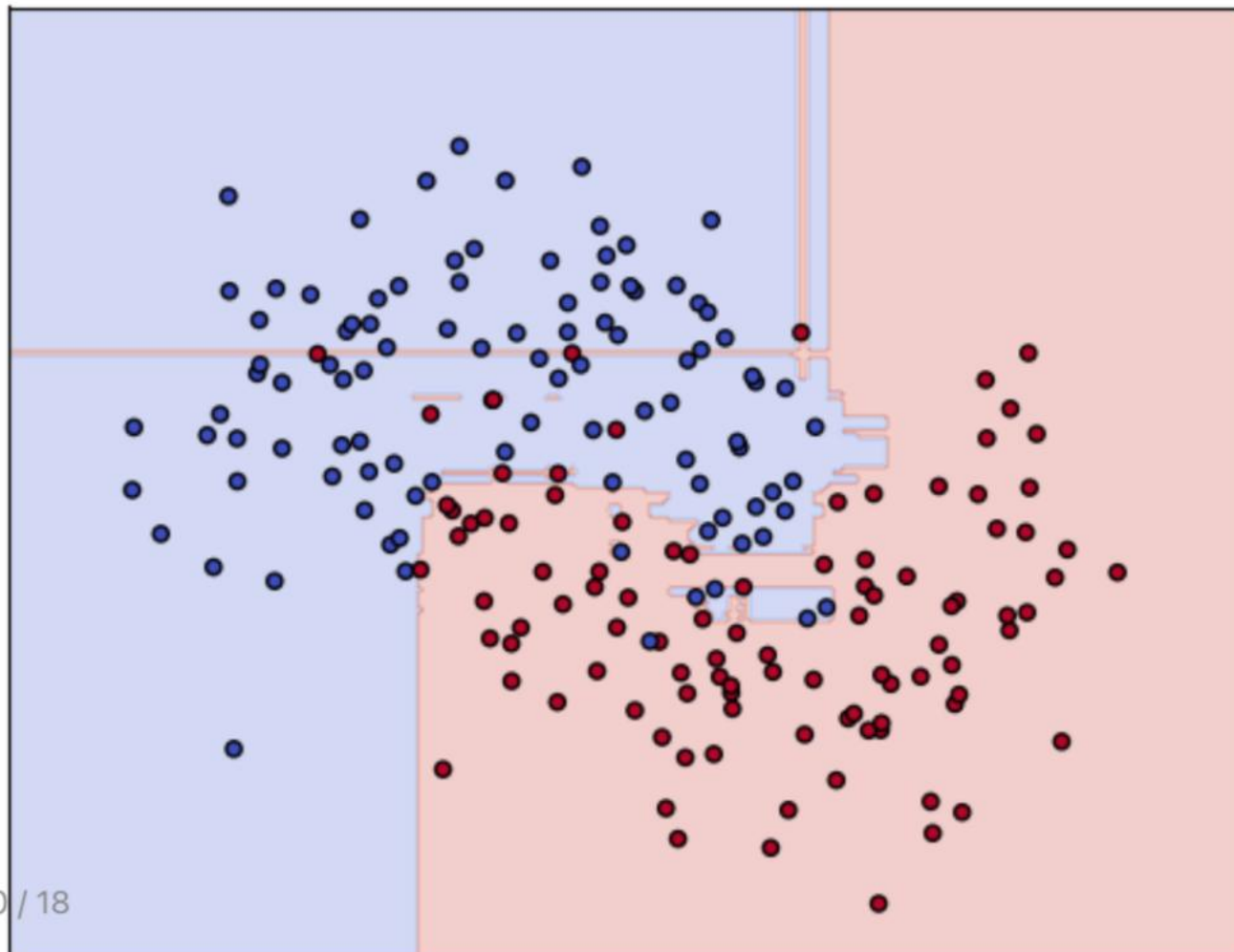
- Bagging + **one extra source of randomness**: at each split, consider only m of the d features
 - Classification: $m \approx \sqrt{d}$. Regression: $m \approx d/3$
- Why: bagging alone still lets the same strong feature dominate every tree - trees stay correlated
- Restricting the feature subset decorrelates the trees, lowering ρ in the variance formula above
- trade-off between the strenght of an individual tree and reducing the correlation

number of trees 20

features per split (m) 2



Random forest, 20 trees, $m=2$



10 / 18





Bagging vs. random forest: bagging introduces randomness in the *data*; random forest also introduces randomness in the *features*.

Boosting

AIM: reducing the bias of weak learners

how → sequentially training weak predictors to improve them!

- Bagging: B trees fit **independently**, in parallel
- Boosting: fit trees **sequentially** - each one focused on the previous ensemble's mistakes
- Weak learners combined into a strong one - here: decision stumps (depth-1 trees)

Deriving AdaBoost from the exponential loss

- Labels $y_i \in \{-1, +1\}$. Ensemble so far: $F_{m-1}(x) = \sum_{k=1}^{m-1} \alpha_k h_k(x)$
- Add one term, greedily, minimizing the exponential loss:

$$(\alpha_m, h_m) = \arg \min_{\alpha, h} \sum_{i=1}^N \exp(-y_i(F_{m-1}(x_i) + \alpha h(x_i)))$$

- Write $w_i^{(m)} = \exp(-y_i F_{m-1}(x_i))$ (fixed at step m , it only depends on the past). The objective becomes

$$\sum w_i^{(m)} \exp(-\alpha y_i h(x_i))$$

Solving for h_m and α_m

- Split the sum into correct / wrong predictions. For any fixed $\alpha > 0$, it is minimized by the h minimizing the **weighted error**

$$\varepsilon_m = \frac{\sum_i w_i^{(m)} 1[h(x_i) \neq y_i]}{\sum_i w_i^{(m)}}$$

- Differentiate w.r.t. α , set to zero:

$$\alpha_m = \frac{1}{2} \ln \frac{1 - \varepsilon_m}{\varepsilon_m}$$

14 / 18

Weight update, $w_i^{(m+1)} = w_i^{(m)} e^{-\alpha_m y_i h_m(x_i)}$: misclassified points reweighted up by e^{α_m} ,  

The AdaBoost algorithm (Freund & Schapire, 1997)

- Initialize $w_i^{(1)} = 1/N$
- For $m = 1, \dots, M$:
 - Fit h_m on the weighted data (minimize ε_m)
 - $\alpha_m = \frac{1}{2} \ln \frac{1-\varepsilon_m}{\varepsilon_m}$
 - Reweight, then renormalize so weights sum to 1
- Output: $F(x) = \text{sign}(\sum_m \alpha_m h_m(x))$

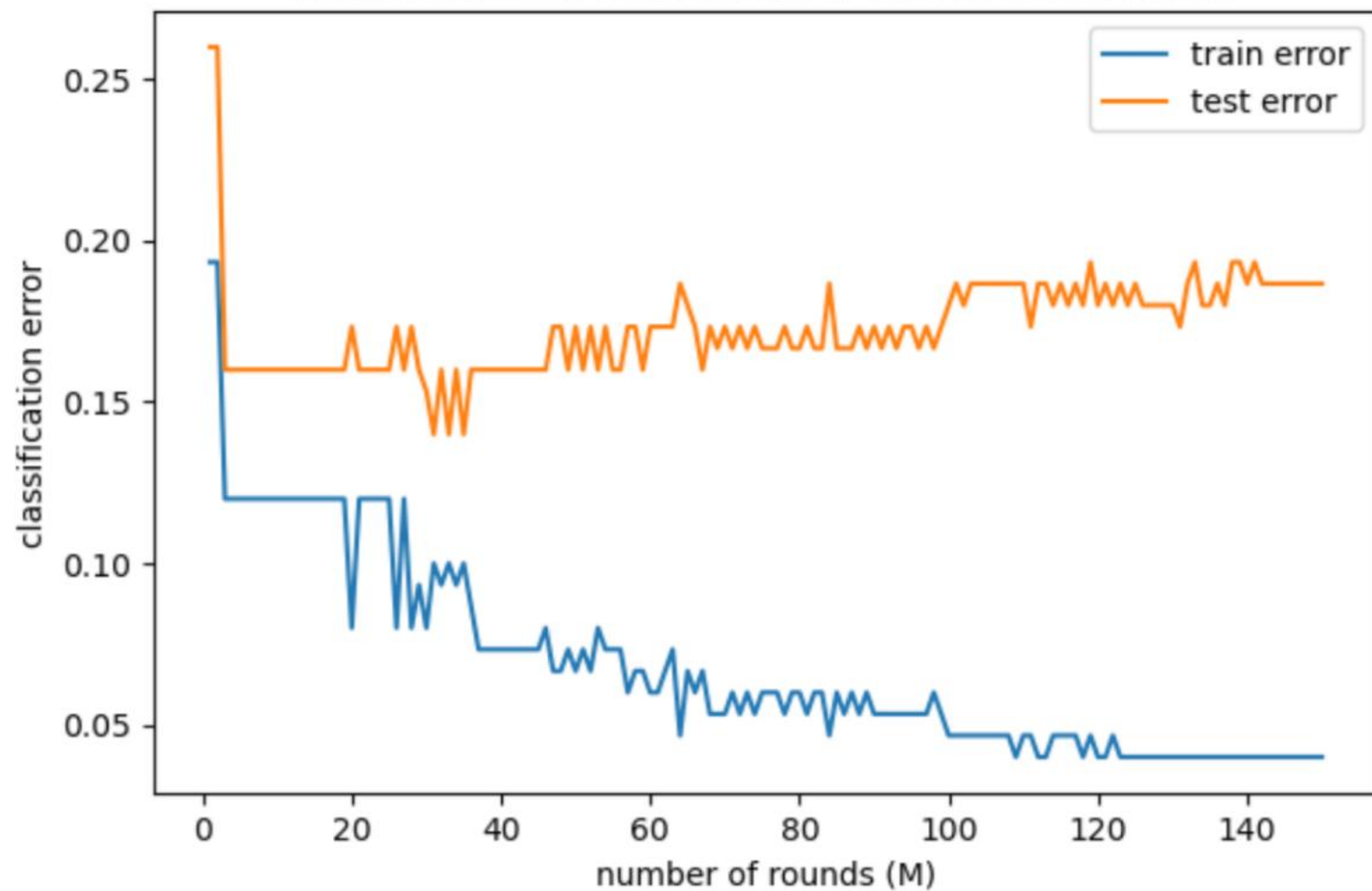


Does more rounds mean overfitting?

- Training error keeps dropping as M grows - does test error follow it back up?
- Below: a noisy Two Moons problem, train and test error against M , on the **same** plot



AdaBoost: train reaches zero, test does not follow





Outlook: gradient boosting

- AdaBoost = forward stagewise fitting, specifically **of the exponential loss**
- Gradient boosting: same recipe, any differentiable loss - fit each new tree to (minus) the loss gradient of the current ensemble
- AdaBoost is recovered as the special case of the exponential loss
- XGBoost, LightGBM, CatBoost

